MITRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Faculty Name	Sanjeev Khatri		
Subject Name with Code	4AD4-06 Introduction to Java Programming		
Semester	4th	**Session**	2022-23 (Even)
Branch	AIDS	**Batch**	A
Unit No.	4	**Submission Date**	24-Mar-23

Lecture No.	*Topic Covered*	*Page No.*	*Total Pages*
1	The Java thread model	1 - 1	1
2	The main thread	2 - 2	1
3	Creating thread	3 - 6	4
4	Creating multiple threads	7 - 9	3
5	Using isAlive () and join ()	10 - 13	4
6	Thread priorities	14 -17	4
7	Synchronization	18 - 23	6
8	Inter thread communications	24 - 26	3
9	Suspending and resuming threads	27 - 30	4
10	Stopping threads	31 -34	4

Faculty

HOD

24/03/2023

Dean Academics

UNIT-4 (MULTITHREADED PROGRAMMING)

LECTURE - 1

The Java Thread Model

Multithreaded programming is one of the many innovative features of Java. A multithreaded program contains two or more parts that can run concurrently. Each part of such a program is called a thread, and each thread defines a separate path of execution. That's why, multithreading is a specialized form of multitasking.

Multithreading Fundamentals

Multi tasking is supported by OS. There are two distinct types of multitasking: process-based and thread-based. Process-based multitasking is the feature that allows our computer to run two or more programs concurrently. In a thread-based multitasking environment, the thread is the smallest unit of dispatchable code. This means that a single program can perform two or more tasks at once.

Advantage of Multithreading

A principal advantage of multithreading is that it enables us to write very efficient programs because it lets us utilize the idle time that is present in most programs.

Thread Model

In a single-core system, concurrently executing threads share the CPU, with each thread receiving a slice of CPU time. Therefore, in a single-core system, two or more threads do not actually run at the same time, but idle CPU time is utilized. It is important to understand that Java's multithreading features work in both types of systems i.e. single-core and multi-core system. In multi-processor/multi-core systems, it is possible for two or more threads to actually execute simultaneously. This can further improve program efficiency and increase the speed of certain operations.

Thread States

A thread can be in one of several states. It can be ***running***. It can be ready to run as soon as it gets CPU time. A running thread can be ***suspended***, which is a temporary halt to its execution. It can later be resumed. A thread can be ***blocked*** when waiting for a resource. A thread can be ***terminated***, in which case its execution ends and cannot be resumed.

MITRC

LECTURE - 2

The Main Thread

All processes have at least one thread of execution, which is usually called the **main thread**, because it is the one that is executed when our program begins. So that, the main thread is the thread that all of the preceding example programs in the book have been using. From the main thread, we can create other threads.

Although the main thread is created automatically when our program is started, it can be controlled through a **Thread** object. To do so, we must obtain a reference to it by calling the method **currentThread( ).** This method returns a reference to the thread in which it is called. Once we have a reference to the main thread, we can control it just like any other thread.

Example-

```
public class FirstThreadDemo
{
    public static void main(String args[])
    {
        Thread t = Thread.currentThread();
        System.out.println("Current thread: " + t);
        // change the name of the thread
        t.setName("My Thread");
        System.out.println("After name change: " + t);
        for(int n = 5; n > 0; n--)
        {
            System.out.println(n);
        }
    }
}
```

Output-

```
Current thread: Thread[main,5,main]
After name change: Thread[My Thread,5,main]
5
4
3
2
1
```

LECTURE - 3

Creating Thread

We create a thread by instantiating an object of type **Thread**. The **Thread** class encapsulates an object that is runnable. As mentioned, Java defines two ways in which we can create a runnable object:

- We can implement the Runnable interface.
- We can extend the Thread class.

Runnable interface

The Runnable interface abstracts a unit of executable code. We can construct a thread on any object that implements the Runnable interface. Runnable defines only one method called **run( )**, which is declared like :

public void run( )

Inside **run( )**, we define the code that constitutes the **new thread**. It is important to understand that **run( )** can call other methods, use other classes, and declare variables just like the main thread. The only difference is that **run( )** establishes the entry point for another, concurrent thread of execution within our program. This thread will end when **run( )** returns.

After creating a class that implements **Runnable**, we will instantiate an object of type **Thread** on an object of that class. **Thread** defines several constructors. Once created, the new thread will not start running until we call its **start( )** method, which is declared within **Thread**. In essence, **start( )** executes a call to **run( )**.

Example-

```
//Create a thread by implementing Runnable.
class MyThread implements Runnable
{
    String thrdName;
    MyThread(String name)
    {
        thrdName = name;
    }

    //Entry point of thread.
    public void run()
    {
        System.out.println(thrdName + " starting.");
        try
        {
            for(int count=0; count<5;count++)
```


MITRC

```
            {
                Thread.sleep(400);
                System.out.println("In " + thrdName + ", count is " + count);
            }
        }
        catch(InterruptedException exc)
        {
            System.out.println(thrdName + " interrupted.");
        }
        System.out.println(thrdName + " terminating.");
    }
}

public class UseThreads {
    public static void main(String args[])
    {
        MyThread mt = new MyThread("Child #1");
        Thread newThrd = new Thread(mt);
        newThrd.start();

        for(int i=0;i<10;i++)
        {
            System.out.println(".");
            try
            {
                Thread.sleep(100);
            }
            catch(InterruptedException exc)
            {
                System.out.println("Main thread interrupted.");
            }
        }
        System.out.println("Main thread ending");
    }
}
```

Output-

Main thread starting.

.

Child #1 starting.

.

.

.

.

In Child #1, count is 0

.

.

.
.
In Child #1, count is 1
.
Main thread ending
In Child #1, count is 2
In Child #1, count is 3
In Child #1, count is 4
Child #1 terminating.

Extend Thread

When a class extends **Thread**, it must override the **run( )** method, which is the entry point for the new thread. It must also call **start( )** to begin execution of the new thread. It is possible to override other Thread methods, but doing so is not required.

Example-

```
public class ExtendThread {
    public static void main(String args[])
    {
        System.out.println("Main thread starting.");
        NewThread nt = new NewThread("Child #1");
        nt.start();

        for(int i=0;i<10;i++)
        {
            System.out.println(".");
            try
            {
                Thread.sleep(100);
            }
            catch(InterruptedException exc)
            {
                System.out.println("Main thread interrupted.");
            }
        }
        System.out.println("Main thread ending");
    }
}

class NewThread extends Thread //Extending Thread Class

{
    NewThread(String name)
    {
        super(name);
```

```
    }

    //Entry point of thread.
    public void run()
    {
        System.out.println(getName() + " starting.");
        try
        {
            for(int count=0; count<5;count++)
            {
                Thread.sleep(400);
                System.out.println("In " + getName() + ", count is " + count);
            }
        }
        catch(InterruptedException exc)
        {
            System.out.println(getName() + " interrupted.");
        }
        System.out.println(getName() + " terminating.");
    }
}
```

Output-

Main thread starting.
.
Child #1 starting.
.
.
.
.
In Child #1, count is 0
.
.
.
.
In Child #1, count is 1
.
Main thread ending
In Child #1, count is 2
In Child #1, count is 3
In Child #1, count is 4
Child #1 terminating.

LECTURE - 4

Creating Multiple Threads

Our program can have as many threads as it needs.

Example-

```
class ChildThread implements Runnable
{
    Thread thrd;
    ChildThread(String name)
    {
        thrd = new Thread(this, name);
    }

    //A method that creates and starts a thread
    public static ChildThread createAndStart(String name)
    {
        ChildThread chThrd = new ChildThread(name);
        chThrd.thrd.start();
        return chThrd;
    }

    //Entry point of thread
    public void run()
    {
        System.out.println(thrd.getName() + " starting.");
        try
        {
            for(int count=0; count < 3; count++)
            {
                Thread.sleep(400);
                System.out.println("In " + thrd.getName() + ", count is " +
(count+1));
            }
        }
        catch(InterruptedException exc)
        {
            System.out.println(thrd.getName() + " interrupted.");
        }
        System.out.println(thrd.getName() + " terminating.");
    }
}

public class MultipleThreads {
    public static void main(String args[])
    {
```

```
        System.out.println("Main Thread Starting.");
        ChildThread ct1 = ChildThread.createAndStart("Child #1");
        ChildThread ct2 = ChildThread.createAndStart("Child #2");
        ChildThread ct3 = ChildThread.createAndStart("Child #3");

        for(int i=0; i<5;i++)
        {
            System.out.println(".");
            try
            {
                Thread.sleep(100);
            }
            catch(InterruptedException exc)
            {
                System.out.println("Main thread interrupted.");
            }
        }
        System.out.println("Main thread ending.");
    }
}
```

Output-

Main Thread Starting.
.
Child #1 starting.
Child #3 starting.
Child #2 starting.
.
.
.
.
In Child #2, count is 1
In Child #1, count is 1
In Child #3, count is 1
Main thread ending.
In Child #3, count is 2
In Child #1, count is 2
In Child #2, count is 2
In Child #2, count is 3
In Child #3, count is 3
In Child #1, count is 3
Child #1 terminating.
Child #2 terminating.
Child #3 terminating.

In this example we can see, once started, all three child threads share the CPU. As we notice that in this run the threads are started in the order in which they are created and Java is free to schedule the execution of threads in its own way.

Thread Life Cycle

- **New State** –
  - A thread enters the newly created by using a new operator.
  - It is new state or born state immediately after creation. i.e. when a constructor is called the Thread is created but is not yet to run() method will not begin until it start() method is called.
  - After the start() method is called, the thread will go to the next state, Runnable state.
- **Runnable State** –
  - Once we invoke the start() method, the thread is runnable.
  - It is divided into two states: A. **The running state** - When the thread is running state, it assigned by CPU cycles and is actually running. B. **The Queued state** - When the thread is in Queued state, it is waiting in the Queue and competing for its turn to spend CPU cycles. It is controlled by Virtual Machine Controller. When we use yield() method it makes sure other threads of the same priority have chance to run. This method cause voluntary move itself to the queued state from the running state.
- **Runnable State** –

  The blocked state is entered when one of the following events occurs: The thread itself or another thread calls the suspend() method (it is deprecated). The thread calls an object's wait() met. The thread itself calls the sleep() method. The thread is waiting for some I/O operations to complete. The thread will join() another thread.
- **Dead State** –

  A thread is dead for any one of the following reasons: It dies a natural death because the un method exists normally. It dies abruptly because an uncaught exception terminates the run method. To find whether thread is alive i.e. currently running or blocked, we can use isAlive() method. If it returns *true* the thread is alive otherwise dead.

MTRC

LECTURE - 5

Using isAlive() and join()

isAlive()

It is often useful to know when a thread has ended. **Thread** provides two means by which we can determine if a thread has ended. First, we can call **isAlive( )** on the thread.

Syntax-

final boolean isAlive( )

The isAlive( ) method returns true if the thread upon which it is called is still running. It returns false otherwise.

Example-

```
public class MultipleThreads1 {
    public static void main(String args[])
    {
        System.out.println("Main Thread Starting.");
        ChildThread ct1 = ChildThread.createAndStart("Child #1");
        ChildThread ct2 = ChildThread.createAndStart("Child #2");
        ChildThread ct3 = ChildThread.createAndStart("Child #3");

        do
        {
            System.out.println(".");
            try
            {
                Thread.sleep(100);
            }
            catch(InterruptedException exc)
            {
                System.out.println("Main thread interrupted.");
            }
        }while(ct1.thrd.isAlive() || ct2.thrd.isAlive() || ct3.thrd.isAlive());
        System.out.println("Main thread ending.");
    }
}
```

Output-

Main Thread Starting.

.

Child #2 starting.

Child #1 starting.

Child #3 starting.

.
.
.
.
In Child #1, count is 1
In Child #3, count is 1
In Child #2, count is 1
.
.
.
.
In Child #2, count is 2
In Child #1, count is 2
In Child #3, count is 2
.
.
.
.
In Child #3, count is 3
In Child #2, count is 3
In Child #1, count is 3
Child #2 terminating.
Child #3 terminating.
Child #1 terminating.
Main thread ending.

NITRC

Join()

To wait for the child threads to terminate we used isAlive(). But to wait for a thread to finish is to call **join()**.

Syntax-

final void join( ) throws InterruptedException

This method waits until the thread on which it is called terminates. Its name comes from the concept of the calling thread waiting until the specified thread joins it. Additional forms of **join( )** allow us to specify a maximum amount of time that we want to wait for the specified thread to terminate.

Example-

```
class ChildThread implements Runnable

{
    Thread thrd;
    ChildThread(String name)
    {
        thrd = new Thread(this, name);
    }

    //A method that creates and starts a thread
    public static ChildThread createAndStart(String name)
    {
        ChildThread chThrd = new ChildThread(name);
        chThrd.thrd.start();
        return chThrd;
    }

    //Entry point of thread
    public void run()
    {
        System.out.println(thrd.getName() + " starting.");
        try
        {
            for(int count=0; count < 3; count++)
            {
                Thread.sleep(400);
                System.out.println("In " + thrd.getName() + ", count is " +
(count+1));
            }
        }
        catch(InterruptedException exc)
        {
            System.out.println(thrd.getName() + " interrupted.");
        }
        System.out.println(thrd.getName() + " terminating.");
    }
}

public class JoinThreads {
    public static void main(String args[])
    {
        System.out.println("Main Thread Starting.");
        ChildThread ct1 = ChildThread.createAndStart("Child #1");
        ChildThread ct2 = ChildThread.createAndStart("Child #2");
        ChildThread ct3 = ChildThread.createAndStart("Child #3");

        try
        {
            ct1.thrd.join(); //Wait until the specified thread ends.
```

```
            System.out.println("Child #1 joined.");
            ct2.thrd.join(); //Wait until the specified thread ends.
            System.out.println("Child #2 joined.");
            ct3.thrd.join(); //Wait until the specified thread ends.
            System.out.println("Child #3 joined.");
        }
        catch(InterruptedException exc)
        {
            System.out.println("Main thread interrupted.");
        }
        System.out.println("Main thread ending.");
    }
}
```

Output-

```
Main Thread Starting.
Child #2 starting.
Child #3 starting.
Child #1 starting.
In Child #3, count is 1
In Child #2, count is 1
In Child #1, count is 1
In Child #2, count is 2
In Child #1, count is 2
In Child #3, count is 2
In Child #1, count is 3
In Child #3, count is 3
In Child #2, count is 3
Child #3 terminating.
Child #2 terminating.
Child #1 terminating.
Child #1 joined.
Child #2 joined.
Child #3 joined.
Main thread ending.
```

MITRC

LECTURE - 6

Thread Priorities

Each thread has associated with it a priority setting. A thread's priority determines, in part, how much CPU time a thread receives relative to the other active threads. Low priority threads receive little and High priority threads receive a lot. Factors other than a thread's priority also affect how much CPU time a thread receives.

When a child thread is started, it's priority setting is equal to that of its parent thread. We can change a thread's priority by calling **setPriority( )**, which is a member of **Thread**.

Syntax-
final void setPriority(int level)

Here, level specifies the new priority setting for the calling thread. The value of level must be within the range **MIN_PRIORITY** and **MAX_PRIORITY**. We can obtain the current priority setting by calling the **getPriority( )** method of **Thread**.

Syntax-
final int getPriority( )

Thread priority summary-

- Java assigns to each thread a priority that determines how that thread should be treated with respect to the others. Thread priorities are integers that specify the relative priority of one thread to another. As an absolute value, a priority is meaningless, a higher-priority thread doesn't run-priority, any thread faster if it is the only thread than running a lower instead, a thread's priority is used to the next. This term is called a **context switch**. The rules that determine when a context switch takes place are simple:
  - A thread can voluntarily relinquish control. This is done by explicitly yielding, sleeping, or blocking on pending I/O. In this scenario, all other threads are examined, and the highest-priority thread that is ready to run is given the CPU.
  - A thread can be preempted by a higher-priority thread. In this case, a lower-priority thread that does not yield the processor is simply preempted—no matter what it is doing—by a higher-priority thread. Basically, as soon as a higher- priority thread wants to run, it does. This is called preemptive multitasking.
- In cases where two threads with the same priority are competing for CPU cycles, the situation is a bit complicated.

Example-

```
class Priority implements Runnable
{

    int count;
    Thread thrd;

    static boolean stop = false;
    static String currnetName;

    Priority(String name)
    {
        thrd = new Thread(this, name);
        count = 0;
        currnetName = name;
    }

    public void run()
    {
        System.out.println(thrd.getName() + " starting.");
        do
        {
            count++;
            if(currnetName.compareTo(thrd.getName()) != 0)
            {
                currnetName = thrd.getName();
                System.out.println("In " + currnetName);
            }
        }while(stop == false && count < 1000);
        stop = true;
        System.out.println("\n" + thrd.getName() + " terminating.");
    }
}

public class ThreadPriorityDemo {
    public static void main(String args[])
    {
        Priority mt1 = new Priority("High Priority");
        Priority mt2 = new Priority("Low Priority");
        Priority mt3 = new Priority("Normal Priority #1");
        Priority mt4 = new Priority("Normal Priority #2");
        Priority mt5 = new Priority("Normal Priority #3");

        mt1.thrd.setPriority(Thread.NORM_PRIORITY + 2);
        mt2.thrd.setPriority(Thread.NORM_PRIORITY - 2);

        mt1.thrd.start();
        mt2.thrd.start();
        mt3.thrd.start();
```

```
        mt4.thrd.start();
        mt5.thrd.start();

        try
        {
            mt1.thrd.join();
            mt2.thrd.join();
            mt3.thrd.join();
            mt4.thrd.join();
            mt5.thrd.join();
        }
        catch(InterruptedException exc)
        {
            System.out.println("Main thread interrupted.");
        }

        System.out.println("\nHigh priority thread counted to " + mt1.count);
        System.out.println("\nLow priority thread counted to " + mt2.count);
        System.out.println("\n1st Normal priority thread counted to " +
mt3.count);
        System.out.println("\n2nd Normal priority thread counted to " +
mt4.count);
        System.out.println("\n3rd Normal priority thread counted to " +
mt5.count);
    }
}
```

Output-

High Priority starting.
Low Priority starting.
Normal Priority #3 starting.
Normal Priority #2 starting.
Normal Priority #1 starting.
In High Priority
In High Priority
In Normal Priority #2
In Normal Priority #2
In Normal Priority #3
In Normal Priority #2
In High Priority
In Normal Priority #1
In High Priority
In Normal Priority #2
In Normal Priority #2
In Normal Priority #3
In High Priority

In High Priority
In Low Priority
In Normal Priority #1
In Normal Priority #2
In Normal Priority #3

Normal Priority #3 terminating.

Normal Priority #1 terminating.

High Priority terminating.

Low Priority terminating.

Normal Priority #2 terminating.

High priority thread counted to 1000

Low priority thread counted to 1

1st Normal priority thread counted to 2

2nd Normal priority thread counted to 603

3rd Normal priority thread counted to 3

Affect of Operating Sytem's implementation of multitasking in Thread Execution

Aside from a thread's priority setting, the most important factor affecting thread execution is the way the operating system implements multitasking and scheduling. Some operating systems use preemptive multitasking in which each thread receives a time slice, at least occasionally. Other systems use nonpreemptive scheduling in which one thread must yield execution before another thread will execute. In nonpreemptive systems, it is easy for one thread to dominate, preventing others from running.

MITRC

LECTURE - 7

Synchronization

When using multiple threads, it is sometimes necessary to coordinate the activities of two or more. The process by which this is achieved is called **synchronization**. The most common reason for synchronization is when two or more threads need access to a shared resource that can be used by only one thread at a time. For example, when one thread is writing to a file, a second thread must be prevented from doing so at the same time.

Other reason for synchronization is when one thread is waiting for an event that is caused by another thread. In this case, there must be some means by which the first thread is held in a suspended state until the event has occurred. Then, the waiting thread must resume execution.

For this purpose, Java implements an elegant twist on an age-old model of interprocess synchronization and that is the **monitor**. A **monitor** is an object that is used as a mutually exclusive lock, or mutex. Only one thread can own a monitor at a given time. When a thread acquires a lock, it is said to have entered the monitor. All other threads attempting to enter the locked monitor will be suspended until the first thread exits the monitor. These other threads are said to be waiting for the monitor. A thread that owns a monitor can re-enter the same monitor if it so desires.

All objects in Java have a monitor. This feature is built into the Java language itself. Thus, all objects can be synchronized. Synchronization is supported by the keyword **synchronized** and a few well-defined methods that all objects have.

Here, object is a reference to the object being synchronized. A synchronized block ensures that a call to a method that is a member of object occurs only after the current thread has successfully entered object's monitor.

There are two ways that we can synchronize our code. First is by using **Synchonized Methods** and second is **Synchonized Statement**.

Synchronized Methods

We can synchronize access to a method by modifying it with the **synchronized** keyword.

Example-

```
class SumArray
{
    private int sum;
    //Method sumArray() is synchronized
    synchronized int sumArray(int nums[])
```

```
    {
        sum = 0;
        for(int i=0;i<nums.length;i++)
        {
            sum += nums[i];
            System.out.println("Running total for " +
                Thread.currentThread().getName() +
                " is " + sum);
            try
            {
                Thread.sleep(10);
            }
            catch(InterruptedException exc)
            {
                System.out.println("Thread interrupted.");
            }
        }
        return sum;
    }
}

class MyThread implements Runnable
{
    Thread thrd;
    static SumArray sa = new SumArray();
    int a[];
    int answer;

    //Construct a new thread
    MyThread(String name, int nums[])
    {
        thrd = new Thread(this, name);
        a = nums;
    }

    public static MyThread createAndStart(String name, int nums[])
    {
        MyThread myThrd = new MyThread(name, nums);
        myThrd.thrd.start();
        return myThrd;
    }

    public void run()
    {
        System.out.println(thrd.getName() + " starting.");

        answer = sa.sumArray(a);
        System.out.println("Sum for " + thrd.getName() +
            "is " + answer);
```

```
        System.out.println(thrd.getName() + " terminating.");
    }
}

public class SyncMethod {
    public static void main(String args[])
    {
        int a[] = {1, 2, 3, 4, 5};
        MyThread mt1 = MyThread.createAndStart("Child #1", a);
        MyThread mt2 = MyThread.createAndStart("Child #2", a);
        try
        {
            mt1.thrd.join();
            mt2.thrd.join();
        }
        catch(InterruptedException exc)
        {
            System.out.println("Main thread interrupted.");
        }
    }
}
```

Output (by using synchronized)-

Child #1 starting.
Child #2 starting.
Running total for Child #1 is 1
Running total for Child #1 is 3
Running total for Child #1 is 6
Running total for Child #1 is 10
Running total for Child #1 is 15
Running total for Child #2 is 1
Sum for Child #1is 15
Child #1 terminating.
Running total for Child #2 is 3
Running total for Child #2 is 6
Running total for Child #2 is 10
Running total for Child #2 is 15
Sum for Child #2is 15
Child #2 terminating.

Output (without synchronized)-

Child #1 starting.
Child #2 starting.
Running total for Child #1 is 1
Running total for Child #2 is 1

Running total for Child #1 is 3
Running total for Child #2 is 5
Running total for Child #1 is 8
Running total for Child #2 is 11
Running total for Child #2 is 15
Running total for Child #1 is 19
Running total for Child #2 is 24
Running total for Child #1 is 29
Sum for Child #1is 29
Sum for Child #2is 29
Child #1 terminating.
Child #2 terminating.

Synchronized Statement

While creating **synchronized** methods within classes that we create is an easy and effective means of achieving synchronization but it will not work in all cases. Fortunately, the solution to this problem is quite easy: We simply put calls to the methods defined by this class inside a synchronized block.

This is the general form of the **synchronized statement**:

```
synchronized(object)  //Here, object is a reference to the object being synchronized.
{
        // statements to be synchronized
}
```

Example-

```
class SumArray
{
    private int sum;
    int sumArray(int nums[]) //Method sumArray() is synchronized
    {
        sum = 0;
        for(int i=0;i<nums.length;i++)
        {
            sum += nums[i];
            System.out.println("Running total for " +
                Thread.currentThread().getName() +
                " is " + sum);
            try
            {
                Thread.sleep(10);
            }
            catch(InterruptedException exc)
            {
```

```
                System.out.println("Thread interrupted.");
            }
        }
        return sum;
    }
}

class MyThread implements Runnable
{
    Thread thrd;
    static SumArray sa = new SumArray();
    int a[];
    int answer;

    //Construct a new thread
    MyThread(String name, int nums[])
    {
        thrd = new Thread(this, name);
        a = nums;
    }

    public static MyThread createAndStart(String name, int nums[])
    {
        MyThread myThrd = new MyThread(name, nums);
        myThrd.thrd.start();
        return myThrd;
    }

    public void run()
    {
        System.out.println(thrd.getName() + " starting.");

        synchronized(sa)
        {
            answer = sa.sumArray(a);
        }
        System.out.println("Sum for " + thrd.getName() +
            "is " + answer);
        System.out.println(thrd.getName() + " terminating.");
    }
}

public class SyncStatement {
    public static void main(String args[])
    {
        int a[] = {1, 2, 3, 4, 5};
        MyThread mt1 = MyThread.createAndStart("Child #1", a);
        MyThread mt2 = MyThread.createAndStart("Child #2", a);
        try
```

```
        {
            mt1.thrd.join();
            mt2.thrd.join();
        }
        catch(InterruptedException exc)
        {
            System.out.println("Main thread interrupted.");
        }
    }
}
```

Output-
Output will remain same as previous example.

Key Point of a synchronized method

- A synchronized method is created by preceding its declaration with synchronized.
- For any given object, once a synchronized method has been called, the object is locked and no synchronized methods on the same object can be used by another thread of execution.
- Other threads trying to call an inuse synchronized object will enter a wait state until the object is unlocked.
- When a thread leaves the synchronized method, the object is unlocked.

MITRC

LECTURE - 8

Inter Thread Communications

Inter-thread communication or Co-operation is all about allowing synchronized threads to communicate with each other. Co-operation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed. Java supports interthread communication with the **wait( )**, **notify( )**, and **notifyAll( )** methods.

The wait( ), notify( ), and notifyAll( ) methods are part of all objects because they are implemented by the Object class. These methods should be called only from within a **synchronized** context.

wait() Method

Causes current thread to release the lock and wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed. The current thread must own this object's monitor, so it must be called from the synchronized method only otherwise it will throw exception.

Following are the various forms of **wait( )** defined by Object:
final void wait( ) throws InterruptedException
final void wait(long millis) throws InterruptedException
final void wait(long millis, int nanos) throws InterruptedException

The first form waits until notified. The second form waits until notified or until the specified period of milliseconds has expired. The third form allows us to specify the wait period in terms of nanoseconds.

notify() Method

Wakes up a single thread that is waiting on this object's monitor. If any threads are waiting on this object, one of them is chosen to be awakened. The choice is arbitrary and occurs at the discretion of the implementation.

notifyAll() Method

Wakes up all threads that are waiting on this object's monitor.

Here are the general forms for **notify( )** and **notifyAll( )**:
final void notify( )
final void notifyAll( )

A call to **notify( )** resumes one waiting thread. A call to **notifyAll( )** notifies all threads, with the scheduler determining which thread gains access to the object.

Difference between wait and sleep

wait()	sleep()
wait() method releases the lock	sleep() method doesn't release the lock.
wait() is the method of Object class	sleep() is the method of Thread class
wait() is the non-static method	sleep() is the static method
wait() is the non-static method	sleep() is the static method
sleep() should be notified by notify() or notifyAll() methods	after the specified amount of time, sleep is completed

Example (inter thread communication in java)-

```
public class interthreadcomdemo
{
    public static void main(String args[])
    {
        final Customer cst=new Customer();
        new Thread()
        {
            public void run()
            {
                cst.withdraw(15000);
            }
        }.start();

        new Thread()
        {
            public void run()
            {
                cst.deposit(10000);
            }
        }.start();
    }
}

class Customer
{
    int amount=10000;
    synchronized void withdraw(int amount)
    {
        System.out.println("going to withdraw...");
        if(this.amount<amount)
        {
            System.out.println("Less balance; waiting for deposit...");
            try
```

```
            {
                wait();
            }
            catch(Exception e){}
        }
        this.amount-=amount;
        System.out.println("withdraw completed...");
    }
    synchronized void deposit(int amount)
    {
        System.out.println("going to deposit...");
        this.amount += amount;
        System.out.println("deposit completed... ");
        notify();
    }
}
```

Output-

going to withdraw...
Less balance; waiting for deposit...
going to deposit...
deposit completed...
withdraw completed...

Interrupting a Thread

If any thread is in sleeping or waiting state (i.e. **sleep()** or **wait()** is invoked), calling the **interrupt()** method on the thread, breaks out the sleeping or waiting state throwing **InterruptedException**. If the thread is not in the sleeping or waiting state, calling the **interrupt()** method performs normal behaviour and doesn't interrupt the thread but sets the interrupt flag to true.

MITRC

LECTURE - 9

Suspending and Resuming Threads

Sometimes it is useful to suspend execution of a thread. The mechanisms to suspend, stop, and resume threads differ between early versions of Java and more modern versions. A program used **suspend( )**, **resume( )**, and **stop( )**, which are methods defined by Thread, to pause, restart, and stop the execution of a thread.

All these have the following forms:
final void resume( )
final void suspend( )
final void stop( )

A thread must be designed so that the run( ) method periodically checks to determine if that thread should suspend, resume, or stop its own execution.

→suspend(), resume(), stop() methods are No Longer in use
While these methods seem to be a perfectly reasonable and convenient approach to managing the execution of threads, they must no longer be used. Because, the **suspend( )** method of the Thread class was deprecated by Java 2. This was done because **suspend( )** can sometimes cause serious problems that involve deadlock. The **resume( )** method is also deprecated. It does not cause problems but cannot be used without the **suspend( )** method as its counterpart. The **stop( )** method of the Thread class was also deprecated by Java 2.

→Then What to do
The following **example** shows one way to implement our own versions of suspend( ), resume( ), and stop( ):

```
class MyThread implements Runnable
{
    Thread thrd;
    boolean suspended;
    boolean stopped;

    MyThread(String name)
    {
        thrd = new Thread(this, name);
        suspended = false;
        stopped = false;
    }

    public static MyThread createAndStart(String name)
    {
        MyThread myThrd = new MyThread(name);
```

```
        myThrd.thrd.start();
        return myThrd;
    }

    public void run()
    {
        System.out.println(thrd.getName() + " starting.");
        try
        {
            for(int i=1; i < 1000; i++)
            {
                System.out.print(i + " ");
                if((i % 10) == 0)
                {
                    System.out.println();
                    Thread.sleep(250);
                }

                synchronized(this)
                {
                    while(suspended)
                    {
                        wait();
                    }
                    if(stopped)
                    {
                        break;
                    }

                }
            }
        }
        catch(InterruptedException exc)
        {
            System.out.println(thrd.getName() + " interrupted.");
        }
        System.out.println(thrd.getName() + " exiting.");
    }
    //Stop the thread
    synchronized void mystop()
    {
        stopped = true;
        suspended = false;
        notify();
    }

    //Suspend the thread
    synchronized void mysuspend()
    {
```

```
        suspended = true;
    }

    //Resume the thread
    synchronized void myresume()
    {
        suspended = false;
        notify();
    }
}



public class SuspendDemo
{
    public static void main(String args[])
    {
        MyThread mt1 = MyThread.createAndStart("My Thread");
        try
        {
            Thread.sleep(1000);
            mt1.mysuspend();
            System.out.println("Suspending thread.");
            Thread.sleep(1000);

            mt1.myresume();
            System.out.println("Resuming thread.");
            Thread.sleep(1000);

            mt1.mysuspend();
            System.out.println("Suspending thread.");
            Thread.sleep(1000);

            mt1.myresume();
            System.out.println("Resuming thread.");
            Thread.sleep(1000);

            mt1.mysuspend();
            System.out.println("Stopping thread.");
            mt1.mystop();
        }
        catch (InterruptedException exce)
        {
            System.out.println("Main thread Interrupted.");
        }

        try
        {
            mt1.thrd.join();
```

```
        } catch (InterruptedException exc)
        {
            System.out.println("main thread Interrupted");
        }
        System.out.println("Main thread exiting");
    }
}
```

Output-

```
My Thread starting.
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39 40
Suspending thread.
Resuming thread.
41 42 43 44 45 46 47 48 49 50
51 52 53 54 55 56 57 58 59 60
61 62 63 64 65 66 67 68 69 70
71 72 73 74 75 76 77 78 79 80
Suspending thread.
Resuming thread.
81 82 83 84 85 86 87 88 89 90
91 92 93 94 95 96 97 98 99 100
101 102 103 104 105 106 107 108 109 110
111 112 113 114 115 116 117 118 119 120
Stopping thread.
My Thread exiting.
Main thread exiting
```

MITRC

LECTURE - 10

Stopping Threads

The stop() method of thread class terminates the thread execution. Once a thread is stopped, it cannot be restarted by start() method. The stop( ) method of the Thread class was also deprecated by Java 2 but we can create our own version of stop method.

Example-

```
class MyThread implements Runnable
{
    Thread thrd;
    boolean suspended;
    boolean stopped;

    MyThread(String name)
    {
        thrd = new Thread(this, name);
        suspended = false;
        stopped = false;
    }

    public static MyThread createAndStart(String name)
    {
        MyThread myThrd = new MyThread(name);
        myThrd.thrd.start();
        return myThrd;
    }

    public void run()
    {
        System.out.println(thrd.getName() + " starting.");
        try
        {
            for(int i=1; i < 1000; i++)
            {
                System.out.print(i + " ");
                if((i % 10) == 0)
                {
                    System.out.println();
                    Thread.sleep(250);
                }

                synchronized(this)
                {
                    while(suspended)
                    {
```

```
                        wait();
                    }
                    if(stopped)
                    {
                        break;
                    }

                }
            }
        }
        catch(InterruptedException exc)
        {
            System.out.println(thrd.getName() + " interrupted.");
        }
        System.out.println(thrd.getName() + " exiting.");
    }
    //Stop the thread
    synchronized void mystop()
    {
        stopped = true;
        suspended = false;
        notify();
    }

    //Suspend the thread
    synchronized void mysuspend()
    {
        suspended = true;
    }

    //Resume the thread
    synchronized void myresume()
    {
        suspended = false;
        notify();
    }
}

public class SuspendDemo

{
    public static void main(String args[])
    {
        MyThread mt1 = MyThread.createAndStart("My Thread");
        try
        {
            Thread.sleep(1000);
            mt1.mysuspend();
            System.out.println("Suspending thread.");
```

```
            Thread.sleep(1000);

            mt1.myresume();
            System.out.println("Resuming thread.");
            Thread.sleep(1000);

            mt1.mysuspend();
            System.out.println("Suspending thread.");
            Thread.sleep(1000);

            mt1.myresume();
            System.out.println("Resuming thread.");
            Thread.sleep(1000);

            mt1.mysuspend();
            System.out.println("Stopping thread.");
            mt1.mystop();
        }
        catch (InterruptedException exce)
        {
            System.out.println("Main thread Interrupted.");
        }

        try
        {
            mt1.thrd.join();
        } catch (InterruptedException exc)
        {
            System.out.println("main thread Interrupted");
        }
        System.out.println("Main thread exiting");
    }
}
```

Output-

My Thread starting.
1 2 3 4 5 6 7 8 9 10
11 12 13 14 15 16 17 18 19 20
21 22 23 24 25 26 27 28 29 30
31 32 33 34 35 36 37 38 39 40
Suspending thread.
Resuming thread.
41 42 43 44 45 46 47 48 49 50
51 52 53 54 55 56 57 58 59 60
61 62 63 64 65 66 67 68 69 70
71 72 73 74 75 76 77 78 79 80
Suspending thread.

Resuming thread.
81 82 83 84 85 86 87 88 89 90
91 92 93 94 95 96 97 98 99 100
101 102 103 104 105 106 107 108 109 110
111 112 113 114 115 116 117 118 119 120
Stopping thread.
My Thread exiting.
Main thread exiting

→In above example mystop() method is working to stop a thread.

MITRC